Práctico: Algoritmos de Ordenamiento con Java y OpenCode

Consigna general

Para cada ejercicio, el alumno deberá escribir primero un prompt para OpenCode donde explique claramente:

- qué algoritmo de ordenamiento debe utilizar;
- cuál es el razonamiento del algoritmo;
- qué datos recibe el programa;
- qué debe mostrar por pantalla;
- qué comentarios debe incluir el código;
- cómo debe verificarse que el resultado sea correcto.

Luego deberá generar la implementación en Java, probarla y ajustar el prompt si el resultado no es correcto.

Ejercicios

1. Ordenamiento Burbuja paso a paso

Implementar en Java un programa que ordene un arreglo de números enteros usando **Bubble Sort**.

El programa debe mostrar el arreglo original y luego el estado del arreglo después de cada pasada.

El objetivo es que el prompt explique por qué el elemento mayor se va desplazando hacia el final.

---

2. Comparación entre Burbuja y Selección

Implementar dos métodos en Java:

bubbleSort(int[] array)
selectionSort(int[] array)

Ambos deben ordenar el mismo arreglo.

El programa debe contar cuántas comparaciones e intercambios realiza cada algoritmo.

El objetivo es que el alumno razone la diferencia entre comparar vecinos y buscar directamente el menor.

---

3. Ordenamiento por Inserción con arreglo casi ordenado

Implementar **Insertion Sort** sobre un arreglo que ya esté casi ordenado.

Ejemplo:

int[] numeros = {1, 2, 3, 5, 4, 6, 7};

El programa debe mostrar cuántos desplazamientos realiza.

El objetivo es que el prompt explique por qué Insertion Sort funciona bien cuando los datos están casi ordenados.

---

4. Ordenamiento de nombres con Selection Sort

Implementar un programa que ordene un arreglo de nombres alfabéticamente utilizando **Selection Sort**.

Ejemplo:

String[] nombres = {"Lucia", "Ana", "Pedro", "Juan"};

El objetivo es que el alumno razone cómo cambia la comparación cuando se ordenan textos y no números.

---

5. ShellSort explicando los gaps

Implementar **ShellSort** en Java.

El programa debe mostrar el valor del `gap` en cada etapa y el estado del arreglo luego de cada pasada.

El objetivo es que el prompt explique por qué ShellSort mejora a Insertion Sort utilizando saltos.

---

6. Quicksort con primer elemento como pivote

Implementar **Quicksort** usando siempre el primer elemento como pivote.

El programa debe mostrar:

- pivote elegido;
- subarreglo izquierdo;
- subarreglo derecho;
- arreglo final ordenado.

El objetivo es que el prompt relacione el algoritmo con recursividad y divide y vencerás.

---

7. Peor caso de Quicksort

Implementar Quicksort sobre un arreglo ya ordenado.

Ejemplo:

int[] numeros = {1, 2, 3, 4, 5, 6, 7};

El programa debe contar la cantidad de llamadas recursivas realizadas.

El objetivo es que el alumno explique en el prompt por qué elegir siempre el primer elemento como pivote puede generar un mal rendimiento.

---

8. MergeSort paso a paso

Implementar **MergeSort** en Java.

El programa debe mostrar:

- cómo se divide el arreglo;
- cuándo llega a la condición de corte;
- cómo se van fusionando los subarreglos.

El objetivo es que el prompt explique claramente la diferencia entre dividir y fusionar.

---

9. Comparación de algoritmos simples

Implementar en Java los algoritmos:

- Bubble Sort
- Selection Sort
- Insertion Sort

El programa debe ordenar el mismo arreglo con los tres métodos y mostrar:

- arreglo ordenado;
- cantidad de comparaciones;
- cantidad de intercambios o desplazamientos.

El objetivo es que el alumno compare las estrategias y no solo el resultado final.

---

10. Sistema de ranking de puntajes

Implementar un programa en Java que reciba un arreglo de jugadores con nombre y puntaje.

El programa debe ordenar el ranking de mayor a menor puntaje utilizando el algoritmo que el alumno considere más adecuado entre los vistos.

Ejemplo:

Jugador("Ana", 1200)
Jugador("Pedro", 900)
Jugador("Lucia", 1500)

El objetivo es que el prompt justifique la elección del algoritmo y explique por qué esa estrategia es conveniente para el problema.